

Correction du TP 2 d'Interprétation et compilation

Christian Rinderknecht

7 et 8 octobre 2003

1 Questions

1. Reconsidérez la sémantique opérationnelle de la division (sans spécification des erreurs). L'ordre d'évaluation des opérandes n'y est pas spécifié. Réécrivez la règle pour qu'elle force l'évaluation de son second opérande avant le premier. Pour cela, inspirez-vous de la règle de la liaison locale.

L'implantation fidèle de cette nouvelle sémantique est maladroite. Proposez une variation simple du code pour la division qui répond au problème.

2. Ajoutez les fonctions et les applications. Notez bien que le type des valeurs doit maintenant changer. Introduisez les modifications nécessaires. N'oubliez pas d'enrichir le traitement d'erreurs dans les cas où on attend un entier et on obtient en fait une fonction, ainsi que le cas contraire. Testez. (*Gardez une trace de vos cas de test.*) Par exemple :

```
1+2*(if true then let f = fun y -> y+1 in f(3+4) else 5)
```

3. Implantez les fonctions récursives natives. Testez par exemple

```
let rec f = fun n -> ifz n then 1 else n*f(n-1) in f 7
let rec x = x in x
```

4. Soit e l'arbre de syntaxe abstraite correspondant au programme 1 2. Existe-t-il un environnement ρ et une valeur v tels que $\rho \vdash e \rightarrow v$? Quelle différence voyez-vous entre cet exemple et **(fun f $\rightarrow$ f f) (fun f $\rightarrow$ f f)**, donné dans le cours? Vérifiez votre raisonnement en soumettant ces expressions à votre calculatrice.
5. Vérifiez que les constructions **let** $x = e_1$ **in** e_2 et **(fun** $x \rightarrow e_2)$ e_1 sont équivalentes du point de vue de l'évaluation — c'est-à-dire que l'une produit une valeur v si et seulement si l'autre produit également v .
6. Ajoutez au langage des liaisons globales de la forme **let** $x = e$.
7. Implantez la sémantique avec références.

8. Vérifiez que les constructions **let** $x = e_1$ **in** e_2 et **(fun** $x \rightarrow e_2)$ e_1 sont équivalentes du point de vue du typage monomorphe, c'est-à-dire que l'une admet le type τ sous l'environnement Γ si et seulement si l'autre admet aussi τ sous Γ .
9. Peut-on typer les expressions ci-dessous de façon monomorphe et, si oui, avec quels types ?

```

let f = fun x  $\rightarrow$  x in f f
fun f  $\rightarrow$  f f
1 2
let f = fun x  $\rightarrow$  x in let x = f 1 in f (fun x  $\rightarrow$  x + 1)

```

10. Implantez l'axiome FUN-OPT qui restreint l'environnement dans les fermetures aux variables libres de la fonction :

$$\rho \vdash \text{Fun}(x, e) \text{ as } f \rightarrow \text{Clos}(x, e, \rho|\mathcal{L}(f)) \quad \text{FUN-OPT}$$

2 Réponses

2.1 Spécification de l'ordre d'évaluation

Nous répondons ici à la question 1.

Une façon de spécifier que le dénominateur est évalué avant le numérateur est :

$$\frac{\rho \vdash e_2 \rightarrow v_2 \quad x \notin \mathcal{L}(e_1) \quad \rho \oplus x \mapsto v_2 \vdash e_1 \rightarrow v_1}{\rho \vdash \text{BinOp}(\text{Div}, e_1, e_2) \rightarrow v_1/v_2} \text{DIV}$$

L'implantation directe de cette sémantique est maladroite, car il faut produire une variable absente des variables libres de e_1 , et il faut alourdir l'environnement. En pratique, il suffit de changer dans l'évaluateur le **let** ... **and** ... en **let** ... **in** **let** ..., comme le montre

```

let rec eval env e = match e with ...
| BinOp (Div, e1, e2)  $\rightarrow$  let v2 = eval env e2 in let v1 = eval env e1 in v1/v2
| ...

```

2.2 Évaluation des applications

Nous commençons par répondre ici à la question 4.

Par définition $e = \text{App}(\text{Const } 1, \text{Const } 2)$, donc la seule règle d'inférence (définissant la relation d'évaluation) dont la conclusion à la forme de e est

$$\frac{\rho \vdash e_1 \rightarrow \text{Clos}(x, e_0, \rho_0) \quad \rho \vdash e_2 \rightarrow v_2 \quad \rho_0 \oplus x \mapsto v_2 \vdash e_0 \rightarrow v}{\rho \vdash \text{App}(e_1, e_2) \rightarrow v} \text{APP}$$

Il faudrait donc que **Const 1** s'évalue en une fermeture (première prémisse). Or la seule règle qui permette l'évaluation de **Const 1** est

$$\rho \vdash \text{Const } n \rightarrow \text{Int}(n) \quad \text{CONST}$$

Donc la règle APP ne peut finalement s'appliquer, c'est-à-dire qu'il n'existe aucun ρ et v tels que $\rho \vdash e \rightarrow v$ car il n'existe pas d'arbre de preuve avec cette conclusion.

D'autre part, nous avons vu dans le cours que l'évaluation du programme **(fun f → f f) (fun f → f f)** ne terminait pas car l'arbre de preuve n'était pas fini.

En conclusion, le programme 1 2 est essentiellement un terme mal formé, alors que **(fun f → f f) (fun f → f f)** est bien formé mais son évaluation ne termine pas.

2.3 La liaison locale

Cette section répond à la question 5.

Considérons les instances de règles dont les conclusions correspondent respectivement aux expressions **let x = e₁ in e₂** et **(fun x → e₂) e₁**. Pour la première, il n'y a qu'à prendre littéralement la règle LET-IN :

$$\frac{\rho \vdash e_1 \rightarrow v_1 \quad \rho \oplus x \mapsto v_1 \vdash e_2 \rightarrow v_2}{\rho \vdash \text{LetIn}(x, e_1, e_2) \rightarrow v_2} \text{LET-IN}$$

et à y substituer v à v_2 (en prévision de la suite) :

$$\frac{\rho \vdash e_1 \rightarrow v_1 \quad \rho \oplus x \mapsto v_1 \vdash e_2 \rightarrow v}{\rho \vdash \text{LetIn}(x, e_1, e_2) \rightarrow v} \text{LET-IN}$$

Pour la seconde expression, il faut prendre la règle

$$\frac{\rho \vdash e_1 \rightarrow \text{Clos}(x, e_0, \rho_0) \quad \rho \vdash e_2 \rightarrow v_2 \quad \rho_0 \oplus x \mapsto v_2 \vdash e_0 \rightarrow v}{\rho \vdash \text{App}(e_1, e_2) \rightarrow v} \text{APP}$$

et y substituer **Fun**(x, e_2) à e_1 , e_1 à e_2 et v_1 à v_2 :

$$\frac{\rho \vdash \text{Fun}(x, e_2) \rightarrow \text{Clos}(x, e_2, \rho) \quad \rho \vdash e_1 \rightarrow v_1 \quad \rho \oplus x \mapsto v_1 \vdash e_2 \rightarrow v}{\rho \vdash \text{App}(\text{Fun}(x, e_2), e_1) \rightarrow v} \text{APP}$$

Puisque $\rho \vdash \text{Fun}(x, e_2) \rightarrow \text{Clos}(x, e_2, \rho)$ est un axiome, on voit que les deux (instances de) règles sont équivalentes. En particulier, la liaison locale peut toujours se traduire en termes d'abstractions et d'applications ; ce n'est pas une construction fondamentale.